exception is made for origins that are sub-domains of the same site, for example www.example.com can still send requests to api.example.com, but genuine cross-site requests are disallowed. If you need to allow cross-site requests with cookies, then you should not use SameSite cookies.
A complication came in October 2019, when Google announced that its Chrome web browser would start marking all cookies as SameSite=lax by default with the release of Chrome 80 in February 2020. (At the time of writing the rollout of this change has been temporarily paused due to the COVID-19 coronavirus pandemic). If you wish to use cross-domain cookies you must now explicitly opt-out of SameSite protections by adding the SameSite=none and Secure attributes to those cookies, but this can cause problems in some older web browsers (see https://www.chromium.org/updates/same-site/incompatible-clients). Google, Apple, and Mozilla are all becoming more aggressive in blocking cross-site cookies to prevent tracking and other security or privacy issues. It's clear that the future of cookies will be restricted to API requests within the same site and that alternative approaches, such as those discussed in the rest of this chapter, must be used for all other cases.

For non-preflight requests, you can let the request proceed once you have added the basic CORS response headers. To add the CORS filter, navigate to src/main/java/com/manning/apisecurityinaction and create a new file named CorsFilter.java in your editor. Type in the contents of listing 5.1 and click save.

Listing 5.1 CORS filter

```
package com.manning.apisecurityinaction;

import spark.*;
import java.util.*;
import static spark.Spark.*;

class CorsFilter implements Filter {
  private final Set<String> allowedOrigins;

  CorsFilter(Set<String> allowedOrigins) {
```